Finding a Drone Without GPS

Ex1 — Visual Navigation for Drones | project overview

1. The problem

A drone normally knows where it is because it listens to GPS satellites. That signal is weak and easy to break: it can be jammed, spoofed, or simply lost between buildings and hills. The moment it goes, the drone is blind about its own position — even though its camera is still working perfectly.

So the question this project answers is simple to state:

> **Given only the video coming out of a drone — no GPS — can we work out where it is?**

The assignment adds one fair and very reasonable condition: we are allowed to prepare in advance. Before the GPS-less flight, we get a recording of the same area *with* GPS. In other words, we are allowed to build a map first, and then use it.

2. The idea behind the approach

The approach is the same trick a hiker uses with a paper map: you look at the landscape around you, find that same landscape on the map, and read your position off the map.

Step one — build the map (done in advance, on the ground)

Take the earlier flight, where GPS was available. Cut its video into still frames, one per second. For each frame we already know exactly where the drone was, because the flight log tells us. Then, for every frame, the computer picks out a few thousand tiny distinctive spots — a corner of a roof, the end of a road marking, the edge of a tree. These are called *features*. The result is a small library of pictures, each one tagged with a real-world coordinate.

Step two — navigate (done in the air, live, with no GPS)

A new frame arrives from the camera. The computer finds the same kind of distinctive spots in it, and asks: which picture in my library shows the same place? Once it finds the match, it works out exactly how the new picture is shifted relative to the library picture, converts that shift from pixels into metres, and adds it to the library picture's known coordinate. That gives the drone's position.

Two extra ingredients stop this from going badly wrong in practice:

- **A sense of momentum.** A drone cannot teleport. The system keeps track of roughly where the drone should be, based on where it was a second ago and how fast it was moving, and only searches the part of the map that is physically within reach. This kills the classic failure where a car park matches a different, identical-looking car park a kilometre away.
- **Knowing when not to trust itself.** A match is only accepted if enough of the matched spots agree with each other geometrically. If they do not, the system says so and coasts on momentum for a moment rather than reporting a confident, wrong answer.

3. How it was solved — and what went wrong first

A first working version was built exactly as described above. It ran end to end, but its accuracy was poor: it was typically wrong by about 40 metres, and sometimes by 200. Investigating *why* turned out to be the most interesting part of the project, because the two causes were not bugs in the code — they were wrong assumptions.

Problem A: the ruler was wrong

To turn a shift measured in pixels into a distance in metres, you need to know how many metres one pixel covers. The first version calculated this from the camera's field of view and the altitude written in the flight log. But that altitude is measured from the *take-off point*, not from the ground the camera is actually looking at — and the flight log for these drones does not record the camera's tilt angle at all. The ruler was about 36% too short, so every distance came out too small.

The fix: stop guessing and measure it. During the preparation stage we already know the real GPS positions, so we can compare how far the drone truly moved between two frames with how far the picture shifted, and read the true metres-per-pixel straight off the data. As a sanity check, the same number was confirmed independently from the image itself, by measuring the width of parking bays in the footage.

Problem B: the test was measuring the wrong thing

The original experiment used the first 80% of a flight as the map and the last 20% as the test. But on these flights the last stretch is the drone flying *back home over ground it has already covered, facing the opposite direction*. A roof looks completely different when you approach it from the other side — different shadows, different faces of buildings visible — and the matching method used here simply cannot recognise it.

This was confirmed by cheating deliberately: even when the system was *told* the correct answer and handed the right map picture, it still could not match it. So the 40-metre figure was never measuring how well the navigator works. It was measuring a limitation of the matching method.

The fix: test it properly. Hold out every fifth frame as the test set, keep the rest as the map, and forbid the system from matching a test frame against any map frame recorded within two seconds of it, so it cannot cheat by matching its own immediate neighbour.

4. The result

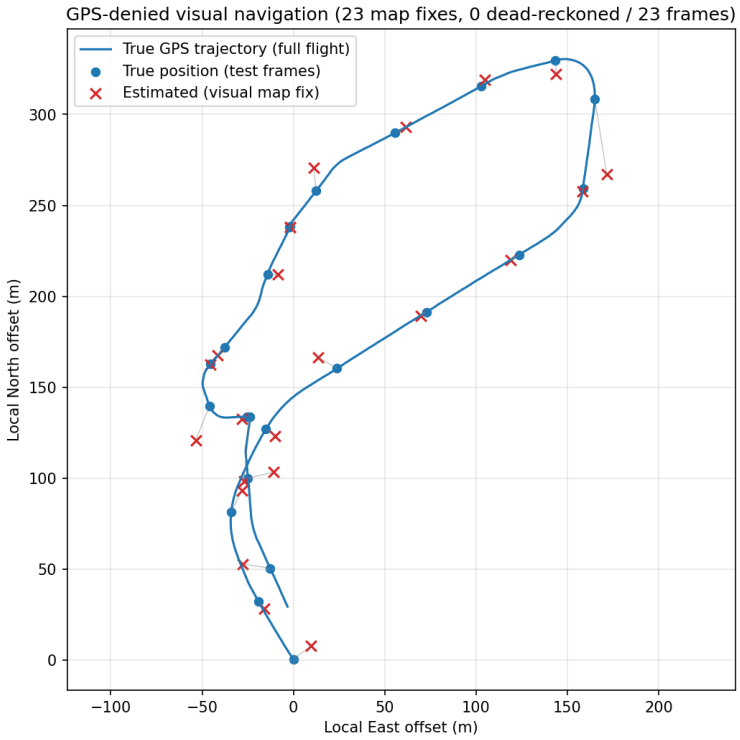
With the ruler corrected and the experiment set up honestly, the system locates the drone from its camera alone to within a few metres — and it produces a confident visual fix on essentially every single frame, instead of frequently giving up:

Flight	Typical error BEFORE	Typical error AFTER	Frames located visually
flight (50 m altitude)	40 m	6 m	23 of 23
flight_0024 (31 m altitude)	17 m	10 m	26 of 27

"Typical error" is the median distance between the estimated position and the true GPS position.

5. Seeing it work

The blue line is the route the drone actually flew, according to its GPS log. The blue dots are the moments we tested. The red crosses are where the system *thought* the drone was, using nothing but the camera. The closer each cross sits to its dot, the better.



Flight over a university campus, roughly 350 m across, flown at about 50 m altitude.

6. What it still cannot do

Honest limitations are worth stating, because they point at the next piece of work:

- **Coming back the other way.** The system still fails to recognise a place it is revisiting from the opposite direction. This is a known weakness of the classical matching method used here. The standard modern remedy is to swap it for a learned, AI-based matcher, which is far better at recognising a place from an unfamiliar angle. The code is arranged so this is a change to one function.
- **One ruler per flight.** The metres-per-pixel figure is a single number for the whole flight. When the camera is tilted, the far side of the picture is genuinely further away than the near side, so the estimate is least accurate towards the edges of the frame.
- **A map is still required.** This is navigation *within a known area*. The drone cannot be dropped somewhere it has never seen. Extending the map from "an earlier flight" to satellite imagery such as Google Earth is the natural next step, and is what the follow-on project targets.

Checking the work. `python summarize.py` re-prints the numbers in this document straight from the stored result files, with nothing to install. To re-run the whole pipeline from scratch, put the flight videos in `data/raw/` and run `./reproduce.sh`. `RESULTS.md` holds the full technical write-up, including the evidence for each of the two problems described above.